॥वसुधैव कुटुम्बकम्॥

Symbiosis Institute of Technology, Pune

Faculty of Engineering

CSE - Academic Year 2026-27

Java Programming – Lab (Batch 2025-29)

Lab Assignment No:- 20

Name of Student	Abobakr mohamed elameen
PRN No.	25070122015
Batch	2025-29
Class	Programming with JAVA
Academic Year & Semester	2nd Year – 3rd Sem
Title of Assignment	Develop a Java application to establish connectivity with a database and perform INSERT, UPDATE, and DELETE operations.
Practice questions	1. Create a Java application to manage employee records using CRUD operations.

```
import java.sql.*;

public class EmployeeCrudDemo {
    static final String URL = "jdbc:mysql://localhost:3306/testdb";
    static final String USER = "root";
    static final String PASSWORD = "XXXXXXX";

    public static void main(String[] args) {
        try (Connection conn = DriverManager.getConnection(URL,
USER, PASSWORD)) {

            // CREATE
            try (PreparedStatement ps = conn.prepareStatement(
                    "INSERT INTO emp_records (id, name, salary)
VALUES (?, ?, ?)")) {
                ps.setInt(1, 301);
                ps.setString(2, "Sneha Rao");
                ps.setDouble(3, 48000.0);
                ps.executeUpdate();
                System.out.println("Employee created");
            }
```

```
            // READ
            readEmployees(conn);

            // UPDATE
            try (PreparedStatement ps = conn.prepareStatement(
                    "UPDATE emp_records SET salary = ? WHERE id =
?")) {
                ps.setDouble(1, 52000.0);
                ps.setInt(2, 301);
                ps.executeUpdate();
                System.out.println("Employee updated");
            }

            readEmployees(conn);

            // DELETE
            try (PreparedStatement ps = conn.prepareStatement(
                    "DELETE FROM emp_records WHERE id = ?")) {
                ps.setInt(1, 301);
                ps.executeUpdate();
                System.out.println("Employee deleted");
            }

            readEmployees(conn);

        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
        }
    }

    static void readEmployees(Connection conn) throws SQLException {
        try (Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery("SELECT id, name,
salary FROM emp_records")) {
            System.out.println("---- Employee Records ---- ");
            while (rs.next()) {
                System.out.println("ID: " + rs.getInt("id")
                        + " | Name: " + rs.getString("name")
                        + " | Salary: " + rs.getDouble("salary"));
            }
        }
    }
}
```

2. Create a Java application to manage student records using CRUD operations. Perform Create, Read, Update and Delete operations on student details such as roll number, name, course and marks using a database.

```
import java.sql.*;
```

```
public class StudentCrudDemo {
    static final String URL = "jdbc:mysql://localhost:3306/testdb";
    static final String USER = "root";
    static final String PASSWORD = "XXXXXXX";

    public static void main(String[] args) {
        try (Connection conn = DriverManager.getConnection(URL,
USER, PASSWORD)) {

            // CREATE
            try (PreparedStatement ps = conn.prepareStatement(
                    "INSERT INTO student_records (roll_no, name,
course, marks) VALUES (?, ?, ?, ?)")) {
                ps.setInt(1, 1);
                ps.setString(2, "abobakr");
                ps.setString(3, "CSE");
                 ps.setInt(4, 95);
                ps.executeUpdate();
                System.out.println("Student created");
            }

            // READ
            readStudents(conn);

            // UPDATE
            try (PreparedStatement ps = conn.prepareStatement(
                    "UPDATE student_records SET marks = ? WHERE
roll_no = ?")) {
                ps.setInt(1, 92);
                ps.setInt(2, 1);
                ps.executeUpdate();
                System.out.println("Student updated");
            }

            readStudents(conn);

            // DELETE
            try (PreparedStatement ps = conn.prepareStatement(
                    "DELETE FROM    student_records WHERE roll_no =
?")) {
                ps.setInt(1, 1);
                ps.executeUpdate();
                System.out.println("Student deleted");
            }

            readStudents(conn);

        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
        }
    }
```

```
    static void readStudents(Connection conn) throws SQLException {
        try (Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery("SELECT roll_no, name,
course, marks FROM student_records")) {
            System.out.println("---- Student Records ---- ");
            while (rs.next()) {
                System.out.println("Roll No: " +
rs.getInt("roll_no")
                        + " | Name: " + rs.getString("name")
                        + " | Course: " + rs.getString("course")
                        + " | Marks: " + rs.getInt("marks"));
            }
        }
    }
}
```

Output

1)

```
C:\Users\abell\OneDrive\Desktop\JAVA_PRACTICALS\JAVA_EXP_20>java -cp .;mysql-connector-j-8.0.33.j
Employee created
----- Employee Records ----
ID: 102 | Name: Rohan Mehta | Salary: 55000.0
Employee updated
----- Employee Records ----
ID: 102 | Name: Rohan Mehta | Salary: 70000.0
Employee deleted
----  Employee Records ----
```

2)

```
C:\Users\abell\OneDrive\Desktop\JAVA_PRACTICALS\JAVA_EXP_20>java -cp .;mysql-connector-j-8.0.33.j
Student created
----  Student Records ----
Roll No: 1 | Name: Abobakr | Course: CSE | Marks: 95
Student updated
----  Student Records ----
Roll No: 1 | Name: Abobakr | Course: CSE | Marks: 100
Student deleted
----  Student Records ----
```

Conclusion	PreparedStatement allows Java applications to safely perform INSERT, UPDATE, and DELETE operations with parameterized queries, protecting against SQL injection while improving performance for repeated queries. Together with SELECT, these operations form complete CRUD functionality — the foundation of nearly every database-driven application, from employee management systems to student record systems.